CSE 2218: Data Structure and Algorithms II Laboratory

Sherajul Arifin
Lecturer, Department of Computer Science and Engineering,
United International **University**

Standard Template Library (STL)

Vector

- Dynamic Array
- Requires **<vector>** header file
- `vector<T> v;`
- `push_back(), pop_back(), size(), resize(), empty(), operator[], at(), front(), back(), begin(), end(), insert(), erase(), clear()`

Reference

Vector

```
#include <iostream>
#include <vector>
using namespace std;
void printVector(vector<int>& v) {
    for (auto x: v) {
        cout << x << " ";
    }
    cout << endl;
}
int main() {
    vector<int> v1 = {1, 4, 2, 3, 5};
    vector<int> v2(5, 9);
    printVector(v1);
    printVector(v2);
    return 0;
}
```

1. Remove all the even elements of a given vector.
   Sample Input:
   1, 2, 3, 2, 5, 6

   Sample Output:
   1, 3, 5

2. Find the second largest element of a vector

Sample Input:

1, 2, 3, 2, 5, 6

Sample Output:

5

[Hint: Use Sorting]

Deque

- **D**ouble **E**nded **QUE**ue
- Requires **<deque>** header file
- `deque<T> da;`
- `push_back(), push_front(), pop_back(), pop_front(), size(), empty(), at(), front(), back(), begin(), end(), insert(), erase(), clear()`

Reference

Deque

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    deque<int> dq = {1, 4, 2, 3, 5};
         // Inserting elements at back and front
    dq.push_back(5);
    dq.push_front(0);
    // Insert element at third position
    auto it = dq.begin() + 2;
    dq.insert(it, 11);
     // Deleting from the back and front
    dq.pop_back();
    dq.pop_front();
    dq.erase(dq.begin());
    for (int i = 0; i < dq.size(); i++)
        cout << dq[i] << " ";
    return 0;
}
```

3. Given a string, you need to find out if the string is a palindrome or not.

Sample Input:

tomato

madam

Sample Output:

NO

YES

4. You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window

Sample Input:

nums = [1,3,-1,-3,5,3,6,7], k = 3

Sample Output:

[3,3,5,5,6,7]

List

- Doubly Linked List (Non-contiguous memory)
- Requires <list> header file
- list<T> l;
- push_back(), push_front(), pop_back(), pop_front(), insert(), erase(), remove(), sort(), reverse(), unique(), merge(), clear(), size(), empty(), front(), back(), begin(), end()

Reference

Set

- Stores unique elements in sorted (ascending) order
- Requires <set> header file
- set<T> s;
- insert(), erase(), find(), count(), lower_bound(), upper_bound(), size(), empty(), clear(), begin(), end()

Reference

5. You are given an array of integers nums. You need to print all the unique integers from the array.

Sample Input:
1, 2, 3, 2, 2, 1, 2, 5, 6

Sample Output:
1, 2, 3, 5, 6

6. You are given an array of integers nums. You need to find if any of the elements of the array is duplicate.

Sample Input:
1, 2, 3, 2, 2, 1, 2, 5, 6

Sample Output:
Duplicate Found

Map

- Stores key-value pairs sorted by key
- Requires <map> header file
- map<Key, Value> m;
- insert({key, val}), m[key] = val, erase(), find(), count(), size(), empty(), clear(),
- Iteration returns keys in ascending order

Reference

7. You are given an array of integers. Print the frequency of the integers.

Sample Input:
1, 2, 3, 2, 2, 1, 2, 5, 6

Sample Output:
1 has appeared 2 time(s)
2 has appeared 4 time(s)
3 has appeared 1 time(s)
5 has appeared 1 time(s)
6 has appeared 1 time(s)

Unordered_map

- Hash Table-based key-value store (no order)
- Requires <unordered_map> header file
- unordered_map<Key, Value> um;
- insert({key, val}), um[key] = val, erase(), find(), count(), size(), empty(), clear()
- Faster average performance than map
- Iteration order is unspecified

Reference

8. Check if Two Strings are Anagram.

Sample Input:
“listen”, “silent”
“door”, “road”

Sample Output:
Anagram
Not Anagram

Sort

- Sorts elements in range [first, last)
- Requires <algorithm> header file
- sort(begin, end) → ascending order
- Custom sort: sort(begin, end, comparator)
- Works only on random access iterators (e.g., vector, array, deque)

Reference

9. Sort an array of pairs using comparator

Binary Search

- Checks if an element exists in a sorted range
- Requires <algorithm> header file
- binary_search(begin, end, value)
- Returns true if value exists, false otherwise
- Time complexity: O(log n)

Reference

10. Check if an Element Exists in the array or not.

Sample Input
1, 3, 5, 7, 9; x = 5
1, 3, 4, 7, 9; x = 5

Sample Output
Exists
Does Not Exist

Lower_bound

- Finds first position where value >= x
- Requires <algorithm> header file
- lower_bound(begin, end, value)
- Returns iterator
- Time complexity: O(log n)
- Use on sorted containers (like vector, set)

Reference

Upper_bound

- Finds first position where value > x
- Requires <algorithm> header file
- upper_bound(begin, end, value)
- Returns iterator
- Time complexity: O(log n)
- Use on sorted containers

Reference

Thank you!